Assessment A-Dice Game with Raspberry Pi

Introduction

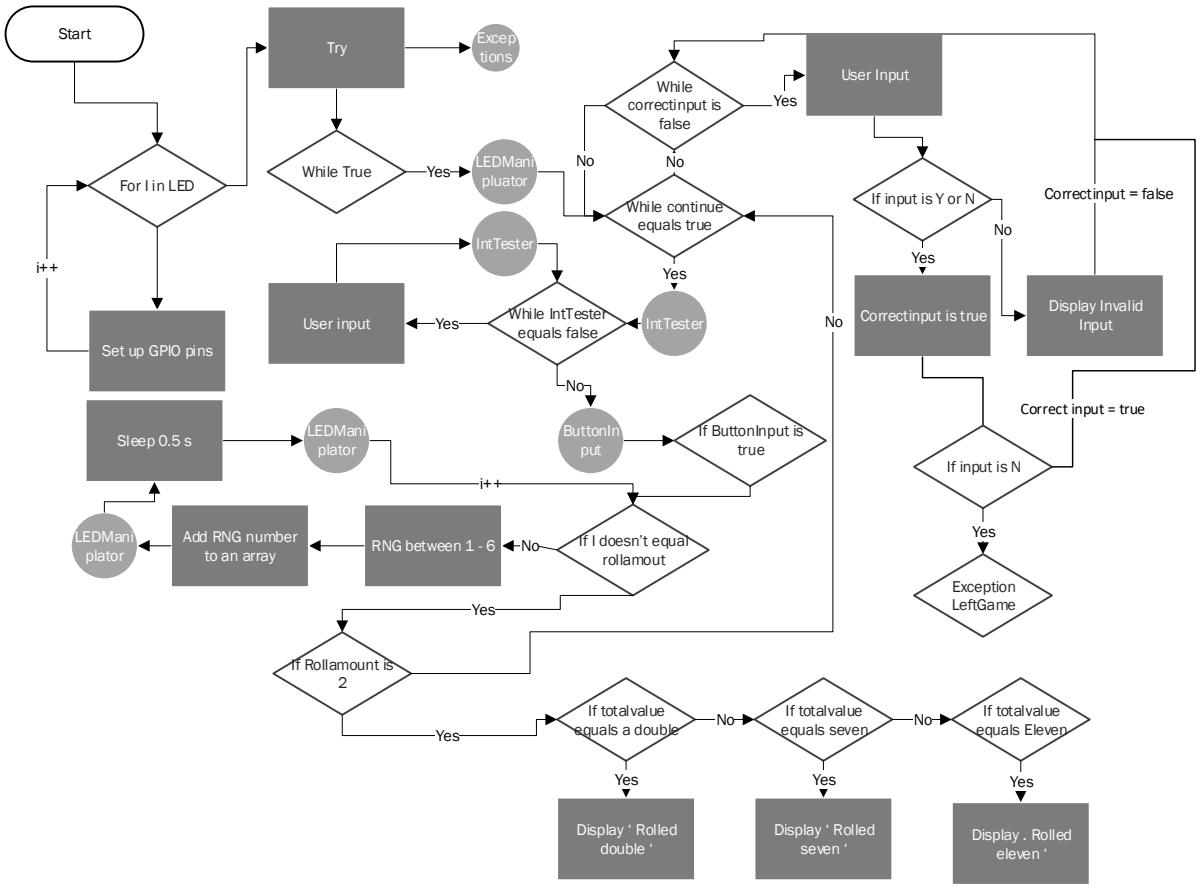
In this assessment the task has been to develop a solution to rolling a dice when the user presses a button on the BerryClip (Blog.soton.ac.uk, 2018) attachment for the RaspberryPi (Raspberry Pi, 2018). In this solution the user is asked how many dice they wish to roll (1 or 2) then simulates this roll when the user presses the button. The user is then presented with both an onscreen and light representation of the random dice roll number. This solution also allows the user to exit the game at any time by pressing Ctrl + C. This solution has additional functionality:

- Notifying when a double, seven or eleven has been thrown.
- Allowing multiple dice to be thrown for one button press
- Telling the user how many doubles, sevens, and elevens were thrown throughout all games, this happens when the user ends the game. Either by pressing Ctrl + C, or leaving when questioned.

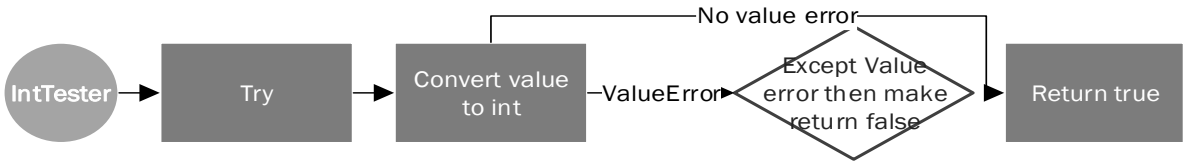
Flow Chart Diagram

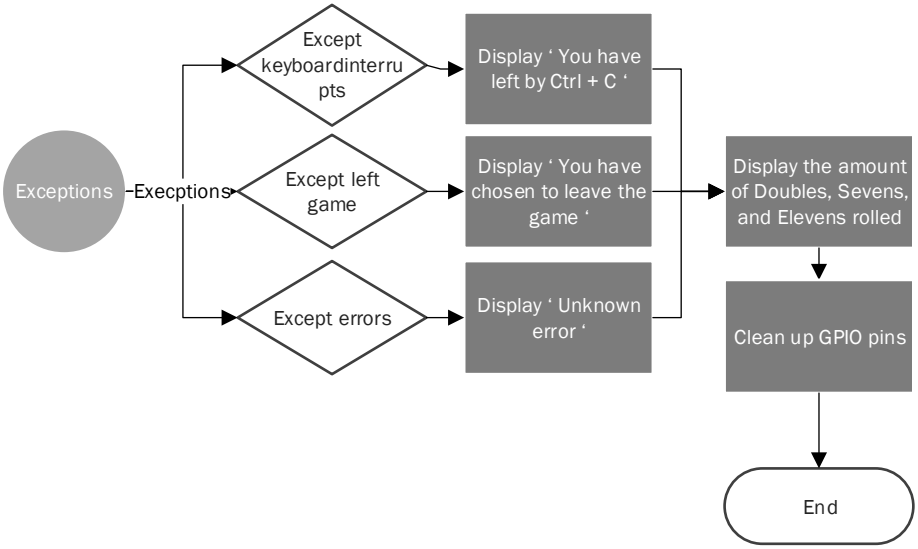
Main code

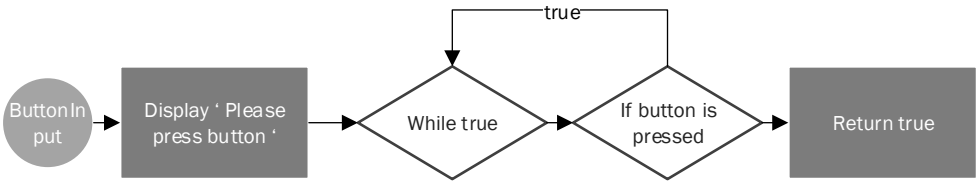
This flowchart diagram shows how the final iteration of the solution works

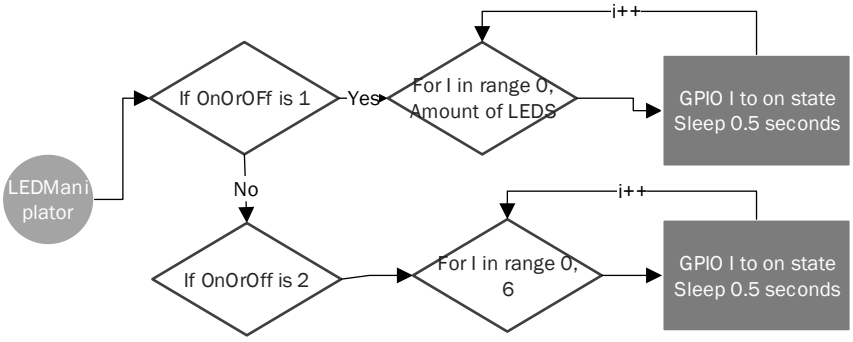


Methods









Implementation

[Complete code under evidence heading]

LEDManiplulator

This methods responsibility is to handle the LED management. This method accomplishes this by accepting 2 arguments, the first to determine if the LEDs are to be set on or off. The second to determine the quantity of LEDs. This method employs 2 if statements and 2 for loops to accomplish its task.

```
#Method used to turn LEDs on or off
#2 arguments used to know whether to turn LEDs off or on, and how many
def LEDManipulator(OnOrOff, AmountOfLEDs):
    if(OnOrOff == 1):                       #This for loop is used to turn LEDs on
        for i in range(0, AmountOfLEDs):    # depending on the seconds argument
            GPIO.output(LED[i], GPIO.HIGH)
            time.sleep(0.5)
    elif(OnOrOff == 2):                     #This for loop is used to turn all LEDs off
        for i in range(0, 6):
            GPIO.output(LED[i], GPIO.LOW)
            time.sleep(0.2)
```

ButtonPress

This method is used to detect button input. This is achieved by using an infinite loop, which has an if function that detects any button presses. Once pressed the loop will end returning true.

```
#Method used to handle button input
def ButtonPress():
    print"Please Press Button"
    while(True):
        if(GPIO.input(Button) == True):     #If button is pushed then method return true
            return True
```

IntTester

This method is used to determine if a value is an integer. This method employs try + exceptions to achieve that.

```
#Method used to test values to see if they are ints
def IntTester(Value):
    try:
        int(Value)
    except ValueError:                      #If values isn't then returns false
        return False
    return True                             #If is int then returns true
```

Exceptions

This section is where are exceptions and game ending information is handled. The solution allows for 2 exceptions.

```
except KeyboardInterrupt:       #Accepts Ctrl + C exception to end solution
    print"You have left by Ctrl + C"
except LeftGame:                #When user opts No when questioned to leave the solution
    print"You have chosen to leave the game"
finally:                        #When an execption is called this will always happen
    print"You have rolled ", rolledDouble, " doubles"       #Displays amount of doubles rolled
    print"You have rolled ", rolledSeven, " sevens"         #Displays amount of sevens rolled
    print"You have rolled ", rolledEleven, " elevens"       #Displays amount of elevens rolled
    LEDManipulator(2, 6)                                    #Turns all LEDs off
    GPIO.cleanup()                                          #Clears GPIO pins
```

Game Setup

This section of code show the starting state of the game.

```
import RPi.GPIO as GPIO
import random
import time
rolledDouble = 0
rolledSeven = 0
rolledEleven = 0
RollAmount = "null"
Continue = True
GPIO.setmode(GPIO.BCM)
LED = [4,17,22,10,9,11] #Saves all LED GPIO pins as array for easy access
Button = 7              #Saves Button as 7 for easy access

#Sets up all LEDs and the button
GPIO.setup(Button, GPIO.IN)
for i in LED:
    GPIO.setup(i, GPIO.OUT)
```

LeftGame

This shows how an additional exception is added to the exception list.

```
#Class used to add LeftGame to Exception list
class LeftGame(Exception):
    pass
```

Main Game

This section is where most of the code is harboured to run the solution. This section handles user input, dice rolling, and the comparing of rolled numbers. This is accomplished by using while loops, if statements, and for loops. Each of these is imperative for the code to function correctly.

```
#Main game loop
try:
    while(True):                            #While loop used to ensure that all the code fulls under the Try
        LEDManipulator(2, 6)                #Turns all LEDs off
        while(Continue == True):
            while(IntTester(RollAmount) == False):      #While loop will also result as false on first loop
                RollAmount = raw_input("How many die do you wish to throw?\n1 or 2\n")
            RollAmount = int(RollAmount)    #Converts user input to an int
            if(ButtonPress() == True):      #Tests for button input by using inline method calling
                DiceNumbers = []            #Creates a new array
                NumberOfRolls = RollAmount
                print"\n"
                for i in range(0, NumberOfRolls):   #For loop used to run the code 1 or 2 times
                    RandNumber = random.randint(1, 6)
                    DiceNumbers.append(RandNumber)  #Adds random number to array
                    LEDManipulator(1, RandNumber)   #Turns the value of RandNumber LEDs on
                    time.sleep(0.5)
                    LEDManipulator(2, 6)            #Turns all LEDs off
                    print"You have rolled a", RandNumber    #Displays random number rolled
                if(NumberOfRolls == 2):             #Tests user selected 2 rolls
                    if(DiceNumbers[0] == DiceNumbers[1]):   #Compares to see if a double
                        print "\nYou have rolled a double", DiceNumbers[1],"\n"
                        rolledDouble = rolledDouble + 1     #Adds on to the rolledDouble
                    if(DiceNumbers[0] + DiceNumbers[1] == 7):   #Compares to see if equals seven
                        print "\nYou have a total of seven\n"
                        rolledSeven = rolledSeven + 1       #Adds one to rolledSeven
                    if(DiceNumbers[0] + DiceNumbers[1] == 11):  #Compares to see if equals eleven
                        print "\nYou have a total of eleven\n"
                        rolledEleven = rolledEleven + 1     #Adds one to rolledEleven

            CorrectInput = False    #Always make sure first loop is false
            while(CorrectInput == False):
                Input = raw_input("Do you wish to go again?\nY or N\n")
                if(Input.upper() == "Y" or Input.upper() == "N"):   #Tests to see if user input is Y or N
                    CorrectInput = True
                else:
                    print"Invalid input\n"                        #If not Y or N then repeats question
            if(Input.upper() == "N"):
                raise LeftGame                                    #If user said N then ends game by raising exception
            else:
                RollAmount = "Repeat"
```

Development

Through the development of this project, the solution has gone through multiple iterations. The major iterations will be highlighted below.

First Iteration

In this iteration, the user could roll 2 dice. However, this code to simulation these di rolls was inefficient using 10's of lines of code, most which was repeating itself. This was quickly fixed in the next iteration. This iterations main purpose was to develop the fundamental concept and mechanics that was needed for the final completion. The concepts and mechanics were the user's ability to choose if they wanted to roll one or two die, and then the mechanic to simulate those die rolls.

```python
try:
    while(True):
        GPIO.cleanup()
#Asks user how many dice they wish to throw
        while(Input == False):
            UserInput = raw_input("How many die do you wish to throw, 1 or 2?   ")
            if(UserInput == "1" or UserInput == "2"):
                                    Input = True
            else:
                                    Input = False
            print"YOU ARE ABOUT TO ROLL" + UserInput + " TIMES"

#Used to simulate dice roll for the first dice. When button is pressed
            if(GPIO.input(7) == 1):
                if(UserInput == "1"):
#Generates random number
                    rThrow = random.randint(1, 6)
#The next section of code turn the relevant lights on. Stating how many was thrown
                    if(rThrow ==1):
                        output(LED[0], high)
                        print("You have rolled ", rThrow)
                        time.sleep(0.5)
                        GPIO.cleanup()
                    if(rThrow == 2):
                        output(LED[0], high)
                        output(LED[1], high)
                        print("You have rolled ", rThrow)
                        time.sleep(0.5)
                        GPIO.cleanup()
                    if(rThrow == 3):
                        output(LED[0], high)
                        output(LED[1], high)
                        output(LED[2], high)
                        print("You have rolled ", rThrow)
                        time.sleep(0.5)
                        GPIO.cleanup()
```

```python
                    if(rThrow == 4):
                        output(LED[0], high)
                        output(LED[1], high)
                        output(LED[2], high)
                        output(LED[3], high)
                        print("You have rolled ", rThrow)
                        time.sleep(0.5)
                        GPIO.cleanup()
                    if(rThrow == 5):
                        output(LED[0], high)
                        output(LED[1], high)
                        output(LED[2], high)
                        output(LED[3], high)
                        output(LED[4], high)
                        print("You have rolled ", rThrow)
                        time.sleep(0.5)
                        GPIO.cleanup()
                        if(rThrow == 6):
                            output(LED[0], high)
                            output(LED[1], high)
                            output(LED[2], high)
                            output(LED[3], high)
                            output(LED[4], high)
                            output(LED[5], high)
                            print("You have rolled ", rThrow)
                            time.sleep(0.5)
                            GPIO.cleanup()

#Similar to the dice roll, except this repeats twice if 2 dice rolls were selected. When button is pressed
            if(UserInput == "2"):
                while(Count < 2):
                    rThrow = random.randint(1, 6)
                    if(rThrow == 1):
                        output(LED[0], high)
                        print("You have rolled ", rThrow)
                        time.sleep(0.5)
                        GPIO.cleanup()
                        Count = Count +1
```

```python
                    if(rThrow == 2):
                        output(LED[0], high)
                        output(LED[1], high)
                        print("You have rolled ", rThrow)
                        time.sleep(0.5)
                        GPIO.cleanup()
                        Count = Count +1
                    if(rThrow == 3):
                        output(LED[0], high)
                        output(LED[1], high)
                        output(LED[2], high)
                        print("You have rolled ", rThrow)
                        time.sleep(0.5)
                        GPIO.cleanup()
                        Count = Count +1
                    if(rThrow == 4):
                        output(LED[0], high)
                        output(LED[1], high)
                        output(LED[2], high)
                        output(LED[3], high)
                        print("You have rolled ", rThrow)
                        time.sleep(0.5)
                        GPIO.cleanup()
                        Count = Count +1
                    if(rThrow == 5):
                        output(LED[0], high)
                        output(LED[1], high)
                        output(LED[2], high)
                        output(LED[3], high)
                        output(LED[4], high)
                        print("You have rolled ",rThrow)
                        time.sleep(0.5)
                        GPIO.cleanup()
                        Count = Count +1
```

```python
                    if(rThrow == 6):
                        output(LED[0], high)
                        output(LED[1], high)
                        output(LED[2], high)
                        output(LED[3], high)
                        output(LED[4], high)
                        output(LED[5], high)
                        print("You have rolled ", rThrow)
                        time.sleep(0.5)
                        GPIO.cleanup()
                        Count = Count +1

#Accepts keyboard interrupts
except KeyboardInterrupt:
        GPIO.cleanup()
```

Second iteration

The second iteration of the project massively improved on the previous iteration. This iteration's advancements on the previous, increased efficient code, telling the user if a 7, 11, or double was thrown. However, this iteration was created by using methods being called inside methods. Which created a 'spaghetti' like code. This meant the user input was being passed through 3 methods as arguments unnecessarily. This would be improved upon in the final iteration.

```
#This method is responsible handling the button press
def ButtonPress(UserInput):
    print("Please press the button\n")
    while(True):
        if(GPIO.input(7) == True):

            DiRoll(UserInput)
            break

#This method tests if a value is an int or not
def IntTester(i):
    try:
        int(i)
    except ValueError:
        return False
    return True

#This method handles the users input
def UserDecision():

    while(True):
        while(Input == False):
            UserInput = raw_input("How many die do you wish to throw, 1 or 2?\n")
#Sends the input to IntTester, if true does following
            if(IntTester(UserInput) == True):
#Tests if user input is 1 or 2
                if(UserInput == "1" or UserInput == "2"):
                    Input = True
                    ButtonPress(UserInput)
                else:
                    Input = False
#After dice roll has happened ask of user wants to continue
        Input = raw_input("Do you wish to roll again?\nY or N\n")
        if(Input.upper() == "Y"):
            UserDecision()
#If no then calls the exception Left
        else:
            raise Left
```

```
#Main game loop
try:
    while(True):
        while(Continue == True):
            UserDecision()


#Execption list
except KeyboardInterrupt:
    print "You have left the game by Ctrl + C"

except Left:
    print "You have chosen to leave the game"
finally:
    GPIO.cleanup()
```

```
for i in LED:
    GPIO.setup(i, GPIO.OUT)

#This is used to set up an additional exception
class Left(Exception):
    pass

#This method is responsible for generating the dice rolls
def DiRoll(x):
    Rollcount = int(x)
#Sets all LEDs to off state
    for i in LED:
        GPIO.output(i, GPIO.LOW)
#For loop runs for how many dice the user wants
    for i in range(0,Rollcount):
        RandomInt = random.randint(1,6)
        print "Dice number is", RandomInt
#Turns relevant LEDs on. Then turns off again
        for i in range(0, RandomInt):
            GPIO.output(LED[i], GPIO.HIGH)
            time.sleep(0.25)
        for i in range(0, RandomInt):
            GPIO.output(LED[i], GPIO.LOW)
            time.sleep(0.25)
```

Final iteration

The goal of iteration 3 was to add the functionality for the program to remember how many doubles, sevens, and elevens were thrown through all games since the program was initially opened, then display this once the user has decided to leave the program (by pressing Ctrl + C or opting to leave once asked). The secondary goals of the iteration were to remove unnecessary methods, and to generally tidy the appearance of the code. [See Implementation for detailed view of code]

Testing

What was tested	Expected Result	Evidence – End of document
Invalid input when asked how many dice wished to be thrown	When an invalid input has been given, user will be notified of this. And will be re asked the question	1.1
Invalid input is asked if they wish to repeat	Refer to previous	1.2
When 1 dice has been selected	One roll will happen	2.1
When 2 die have been selected	Two rolls will happen	2.2
When a double has been thrown	A message will be displayed stating such	3.1
When an eleven has been throw	Refer to previous	3.2
When a seven has been thrown	Refer to previous	3.3
When user selects to not continue playing	Game ends, user told why the game has ended. Then a list of how many doubles, sevens, and elevens have been thrown	4.1
When the user presses Ctrl+C	Refer to previous	4.2
Rolled 1	• 1 LED lights up • Message stating 1 has been throws shows	5.1
Rolled 2	• 1 LED lights up • Message stating 1 has been throws shows	5.2
Rolled 3	• 1 LED lights up • Message stating 1 has been throws shows	5.3
Rolled 4	• 1 LED lights up • Message stating 1 has been throws shows	5.4
Rolled 5	• 1 LED lights up • Message stating 1 has been throws shows	5.5
Rolled 6	• 1 LED lights up • Message stating 1 has been throws shows	5.6

Conclusion

Throughout this project, I have successfully created a solution that can seamlessly roll a user defined amount of dice, storing their value, and manipulate LEDs on a Raspberry pi. Before attempting this project, I had touched upon Python, but never in detail. I also had never operated with a Raspberry Pi nor touched upon affect hardware with software. Although, the developing of this solution has taught me how to affect hardware (GPIO pins) using python, and has taught me various python skills, such as working with definitions.

Bibliography

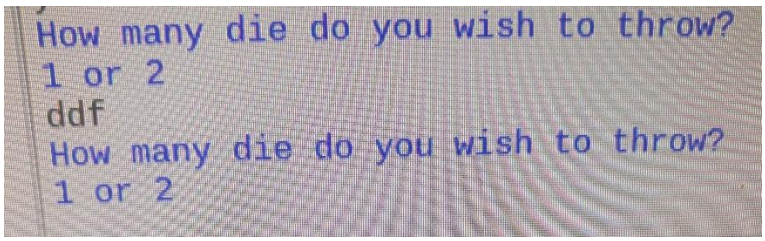
Blog.soton.ac.uk. (2018). *Berry Clip – Raspberry Pi Development Notes*. [online] Available at: https://blog.soton.ac.uk/pi/modules-available/berry-clip/ [Accessed 30 Nov. 2018].

Raspberry Pi. (2018). *What is a Raspberry Pi?*. [online] Available at: https://www.raspberrypi.org/help/what-%20is-a-raspberry-pi/ [Accessed 30 Nov. 2018].

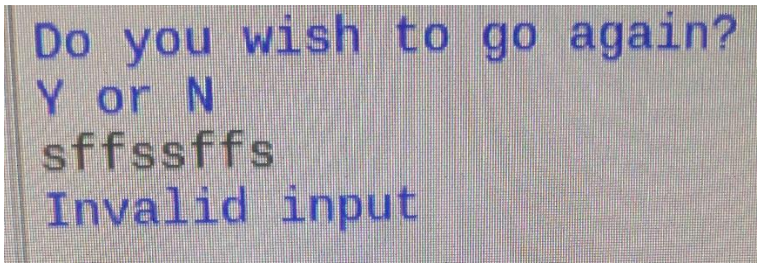
Evidence

1

1.1

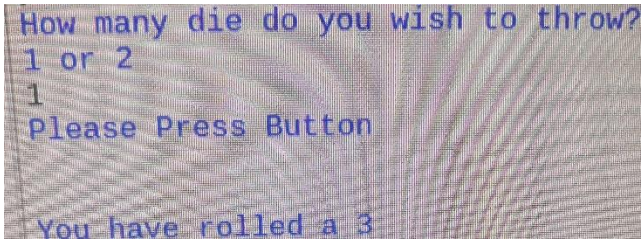


1.2

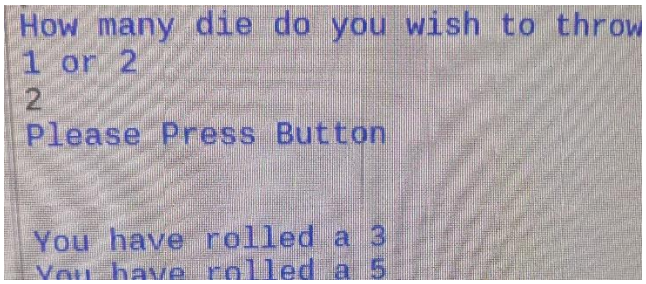


2

2.1

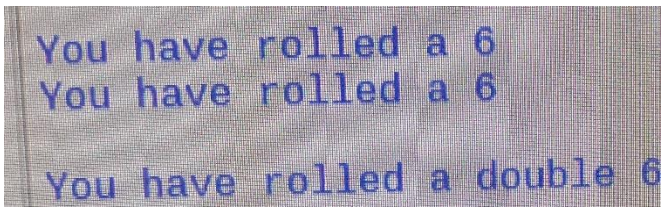


2.2

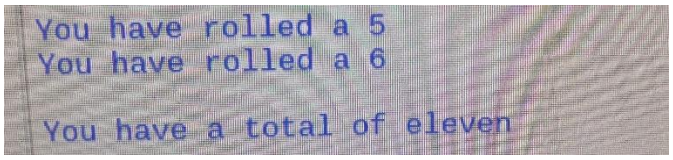


3

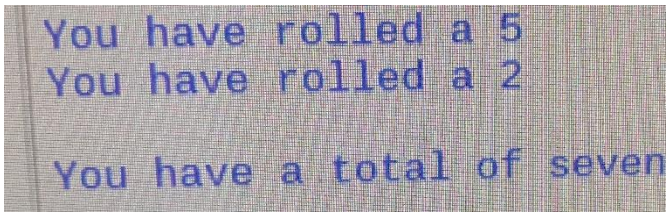
3.1



3.2

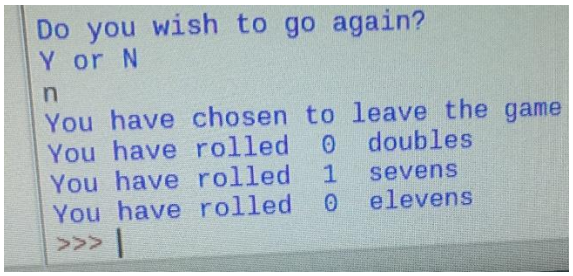


3.3

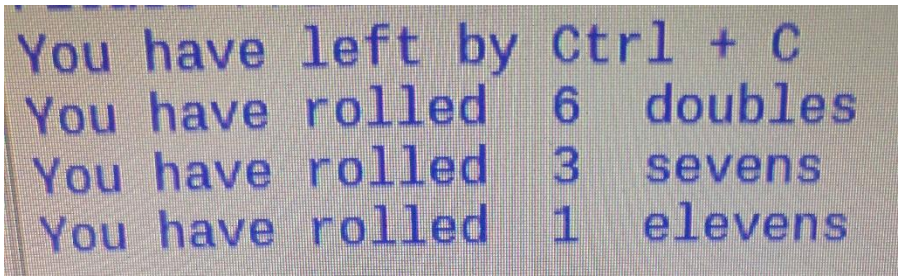


4

4.1

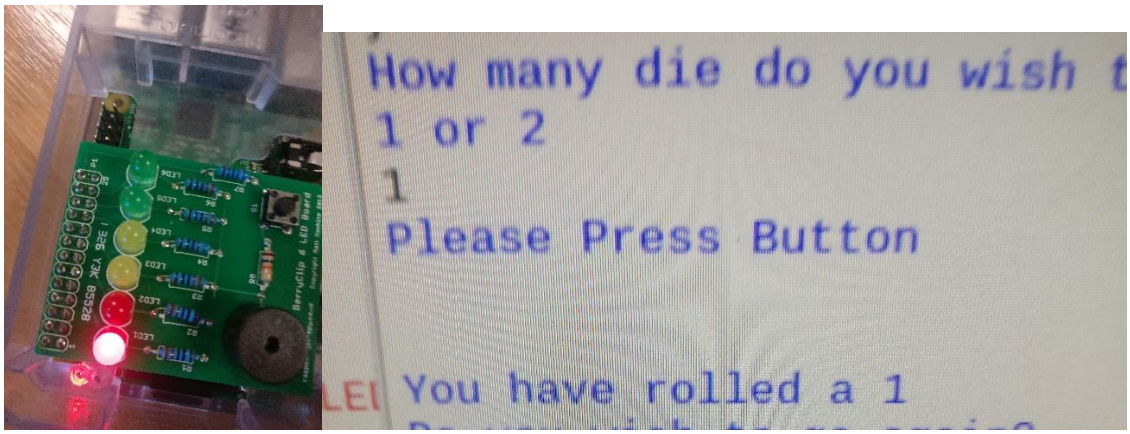


4.2

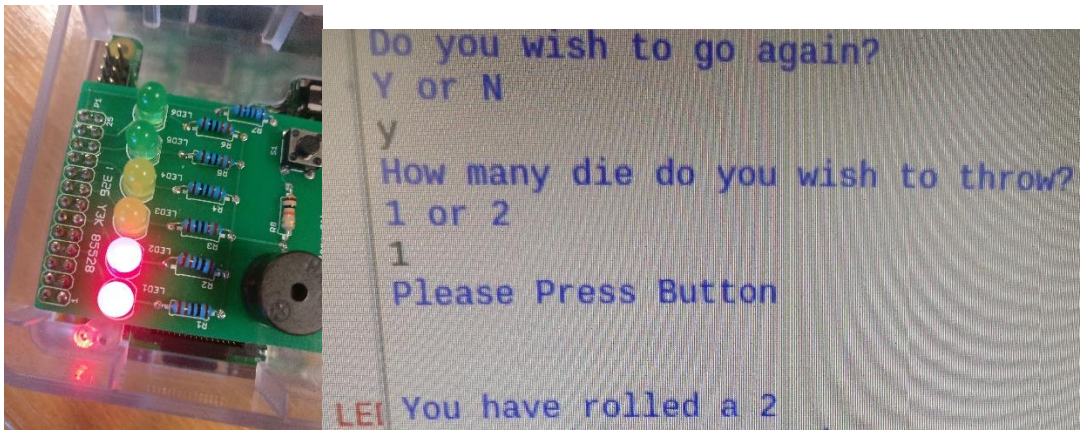


5

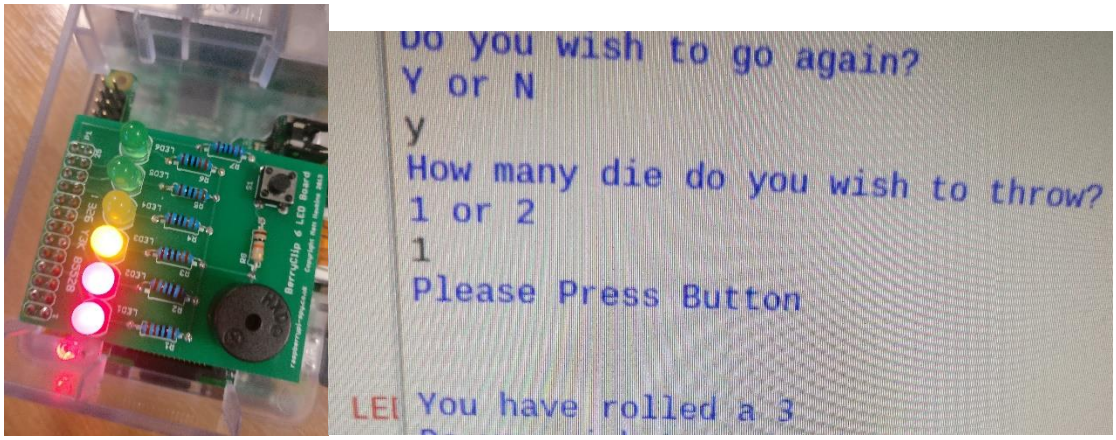
5.1


How many die do you wish t
1 or 2
1
Please Press Button
You have rolled a 1

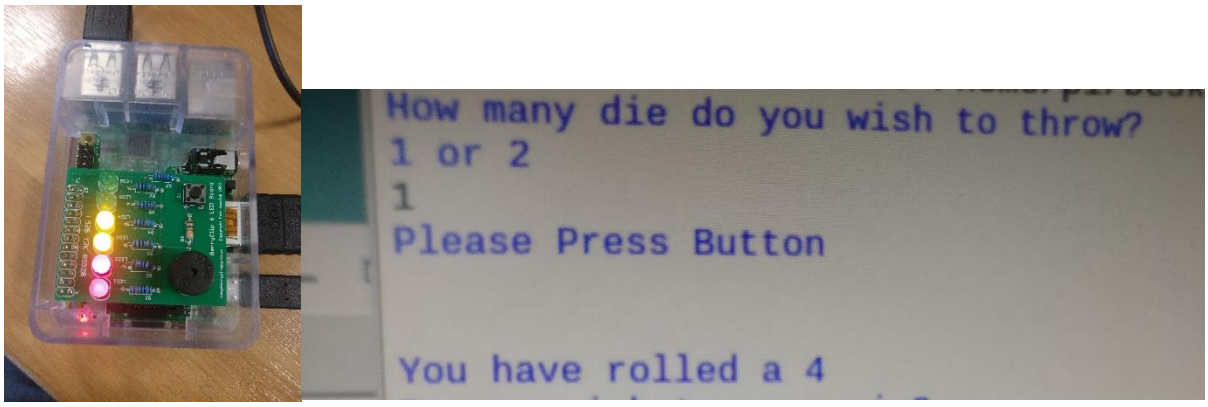
5.2


Do you wish to go again?
Y or N
y
How many die do you wish to throw?
1 or 2
1
Please Press Button
You have rolled a 2

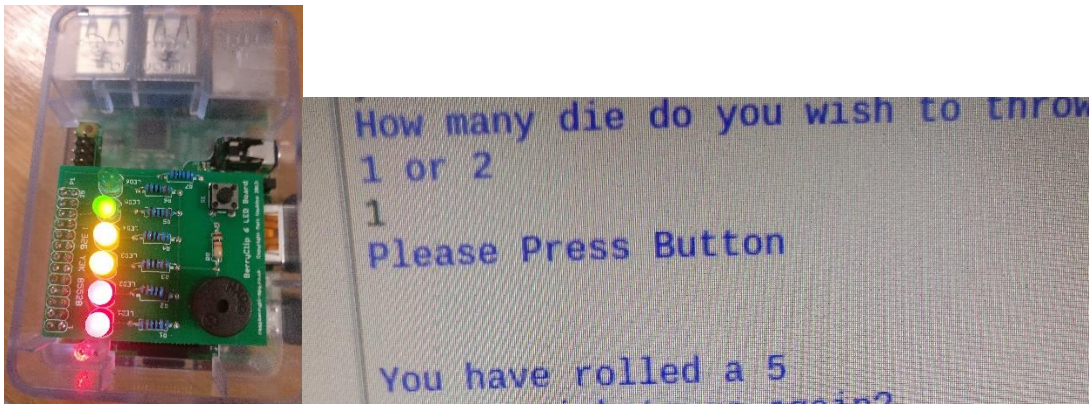
5.3


Do you wish to go again?
Y or N
y
How many die do you wish to throw?
1 or 2
1
Please Press Button
You have rolled a 3

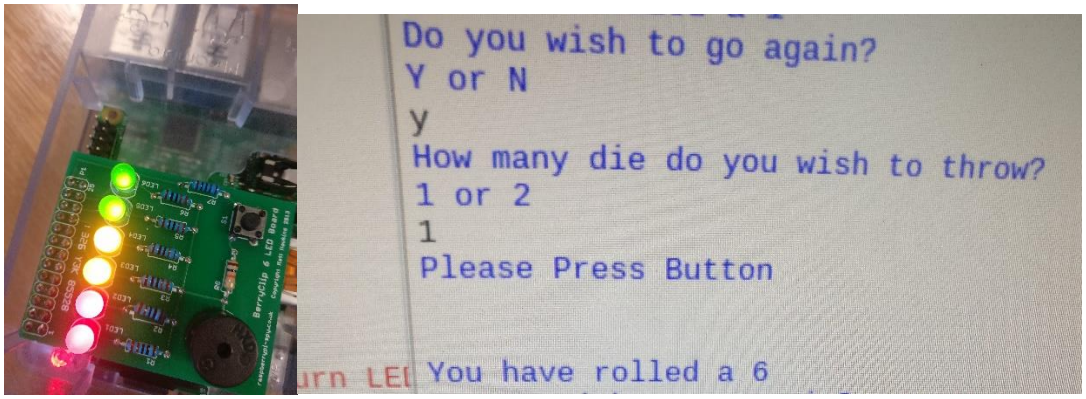
5.4


How many die do you wish to throw?
1 or 2
1
Please Press Button
You have rolled a 4

5.5


How many die do you wish to throw
1 or 2
1
Please Press Button
You have rolled a 5

5.6


Do you wish to go again?
Y or N
y
How many die do you wish to throw?
1 or 2
1
Please Press Button
You have rolled a 6

Complete Final code

```
import RPi.GPIO as GPIO
import random
import time
rolledDouble = 0
rolledSeven = 0
rolledEleven = 0
RollAmount = "null"
Continue = True
GPIO.setmode(GPIO.BCM)
LED = [4,17,22,10,9,11] #Saves all LED GPIO pins as array for easy access
Button = 7              #Saves Button as 7 for easy access

#Sets up all LEDs and the button
GPIO.setup(Button, GPIO.IN)
for i in LED:
    GPIO.setup(i, GPIO.OUT)

#Class used to add LeftGame to Exception list
class LeftGame(Exception):
    pass

#Method used to turn LEDs on or off
#2 arguments used to know whether to turn LEDs off or on, and how many
def LEDManipulator(OnOrOff, AmountOfLEDs):
    if(OnOrOff == 1):                       #This for loop is used to turn LEDs on
        for i in range(0, AmountOfLEDs):    # depending on the seconds argument
            GPIO.output(LED[i], GPIO.HIGH)
            time.sleep(0.5)
    elif(OnOrOff == 2):                     #This for loop is used to turn all LEDs off
        for i in range(0, 6):
            GPIO.output(LED[i], GPIO.LOW)
            time.sleep(0.2)


#Method used to handle button input
def ButtonPress():
    print"Please Press Button"
    while(True):
        if(GPIO.input(Button) == True):     #If button is pushed then method return true
            return True
```

```
#Method used to handle button input
def ButtonPress():
    print"Please Press Button"
    while(True):
        if(GPIO.input(Button) == True):     #If button is pushed then method return true
            return True

#Method used to test values to see if they are ints
def IntTester(Value):
    try:
        int(Value)
    except ValueError:                      #If values isn't then returns false
        return False
    return True                             #If is int then returns true

#Main game loop
try:
    while(True):                            #While loop used to ensure that all the code fulls under the Try
        LEDManipulator(2, 6)                #Turns all LEDs off
        while(Continue == True):
            while(IntTester(RollAmount) == False):      #While loop will also result as false on first loop
                RollAmount = raw_input("How many die do you wish to throw?\n1 or 2\n")
            RollAmount = int(RollAmount)    #Converts user input to an int
            if(ButtonPress() == True):      #Tests for button input by using inline method calling
                DiceNumbers = []            #Creates a new array
                NumberOfRolls = RollAmount
                print"\n"
                for i in range(0, NumberOfRolls):   #For loop used to run the code 1 or 2 times
                    RandNumber = random.randint(1, 6)
                    DiceNumbers.append(RandNumber)  #Adds random number to array
                    LEDManipulator(1, RandNumber)   #Turns the value of RandNumber LEDs on
                    time.sleep(0.5)
                    LEDManipulator(2, 6)            #Turns all LEDs off
                    print"You have rolled a", RandNumber    #Displays random number rolled
                if(NumberOfRolls == 2):             #Tests user selected 2 rolls
                    if(DiceNumbers[0] == DiceNumbers[1]):   #Compares to see if a double
                        print "\nYou have rolled a double", DiceNumbers[1],"\n"
                        rolledDouble = rolledDouble + 1     #Adds on to the rolledDouble
                    if(DiceNumbers[0] + DiceNumbers[1] == 7):   #Compares to see if equals seven
                        print "\nYou have a total of seven\n"
                        rolledSeven = rolledSeven + 1       #Adds one to rolledSeven
```

```
                    if(DiceNumbers[0] + DiceNumbers[1] == 11):  #Compares to see if equals eleven
                        print "\nYou have a total of eleven\n"
                        rolledEleven = rolledEleven + 1     #Adds one to rolledEleven

            CorrectInput = False    #Always make sure first loop is false
            while(CorrectInput == False):
                Input = raw_input("Do you wish to go again?\nY or N\n")
                if(Input.upper() == "Y" or Input.upper() == "N"):   #Tests to see if user input is Y or N
                    CorrectInput = True
                else:
                    print"Invalid input\n"                        #If not Y or N then repeats question
            if(Input.upper() == "N"):
                raise LeftGame                                    #If user said N then ends game by raising exception
            else:
                RollAmount = "Repeat"

except KeyboardInterrupt:       #Accepts Ctrl + C exception to end solution
    print"You have left by Ctrl + C"
except LeftGame:                #When user opts No when questioned to leave the solution
    print"You have chosen to leave the game"
finally:                        #When an execption is called this will always happen
    print"You have rolled ", rolledDouble, " doubles"       #Displays amount of doubles rolled
    print"You have rolled ", rolledSeven, " sevens"         #Displays amount of sevens rolled
    print"You have rolled ", rolledEleven, " elevens"       #Displays amount of elevens rolled
    LEDManipulator(2, 6)                                     #Turns all LEDs off
    GPIO.cleanup()                                           #Clears GPIO pins
```